

Cardiff Metropolitan University | Prifysgol Metropolitan Caerdydd

SEN5001

Mobile and Web Technologies

WRIT1

By James Harris

ST20190404

St20190404@outlook.cardiffmet.ac.uk

November 2024

Wales, United Kingdom

Table of Contents

Abstract..... 3
Introduction..... 4
Proposed Design and Architecture.....6
Wireframes: 6
Development Process..... 16
Implementation: 16
Challenges: 27
Testing: 29
Conclusion & Future Work.....38
References.....39

Abstract

This report details the design, development, and implementation of the Welsh Roots mobile app, created to provide users with an engaging and educational platform for exploring Wales's cultural identity, history, and heritage. The app serves as a digital gateway, offering features such as virtual museum tours, exhibit highlights, an extensive content library, bilingual support and interactive functionalities, including user reviews and a search tool. It outlines the technical implementation of key components, including Firebase for authentication, Google Maps for location-based services, and SharedPreferences for user preferences. This report also evaluates the app's focus on accessibility and personalisation through features such as light and dark themes, and Welsh and English language support, catering to a diverse user base as well as how accessible the app is to individuals with disabilities. Furthermore, it examines the challenges encountered during development, such as emulator compatibility issues, and discusses the troubleshooting strategies employed to resolve them. By reflecting on the development process and the app's core functionality, this report provides an in-depth analysis of how the Welsh Roots app effectively promotes and preserves Welsh culture while delivering an intuitive and inclusive user experience.

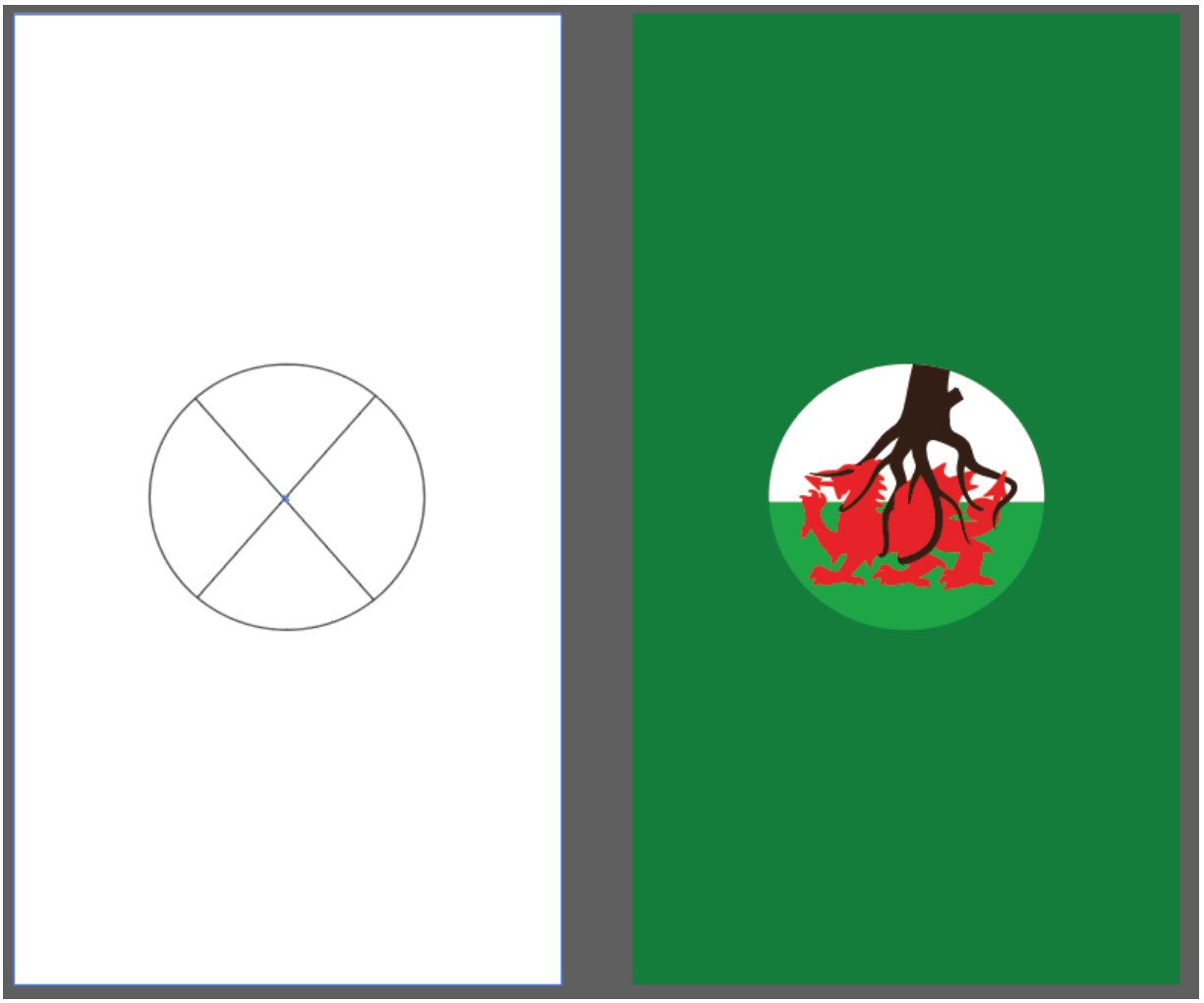
Introduction

Apps are a popular tool for education, especially in learning a new language or culture. Mobile apps are one of the default formats that people expect to have available when approaching a new language skill (Ceci, 2024). There are many popular language learning apps such as Duolingo, Babel, Lingutown and many more. The reported statistics from statista as of November 2024 states that in August of 2024 there were 26.5 million downloads of leading language learning apps from global learners (Ceci, 2024). The market for language learning apps is big and open to innovation. The only problem I have found with apps that just teach the language skill is that they leave out one of the most important parts of learning a new language which is learning a new culture. The app I chose to design is called Welsh Roots. It is an app centred around teaching Welsh history and Welsh culture in an attempt to fill that gap. The app will accomplish this by showcasing the many Welsh events that are celebrated in Welsh culture and highlighting Welsh museums and exhibitions that focus on specific aspects of Welsh history. Additionally, the app will provide the locations of these museums, enabling users to plan travel routes and organise trips to Wales to visit these museums and get the most out of there visit to Wales. The goal of this app is to offer people the opportunity to learn about Wales, Welsh culture, Welsh heritage, and Welsh history. The target audience includes teachers, children, language learners, and individuals or groups with an interest in Welsh culture. Welsh Roots meets the assignment requirements by focusing on Welsh culture and heritage, with a specific emphasis on museums across Wales. The app highlights four key museums: National Waterfront Museum, Big Pit, St. Fagans, and National Museum Cardiff. It also includes content that delves into Welsh folklore and history, ensuring the material is culturally respectful and educational. To fulfil the requirement for virtual museum tours, the app features tours from three Welsh museums: Big Pit, St. Fagans, and National Museum Cardiff. These tours are implemented through YouTube videos, supplemented by Google Street View, which allows users to explore both the surrounding areas and the interiors of the museums. The app also meets the exhibit highlight requirement by showcasing exhibitions from each museum, complete with detailed descriptions and videos. Users are provided with the opportunity to engage with the app further by leaving reviews about their

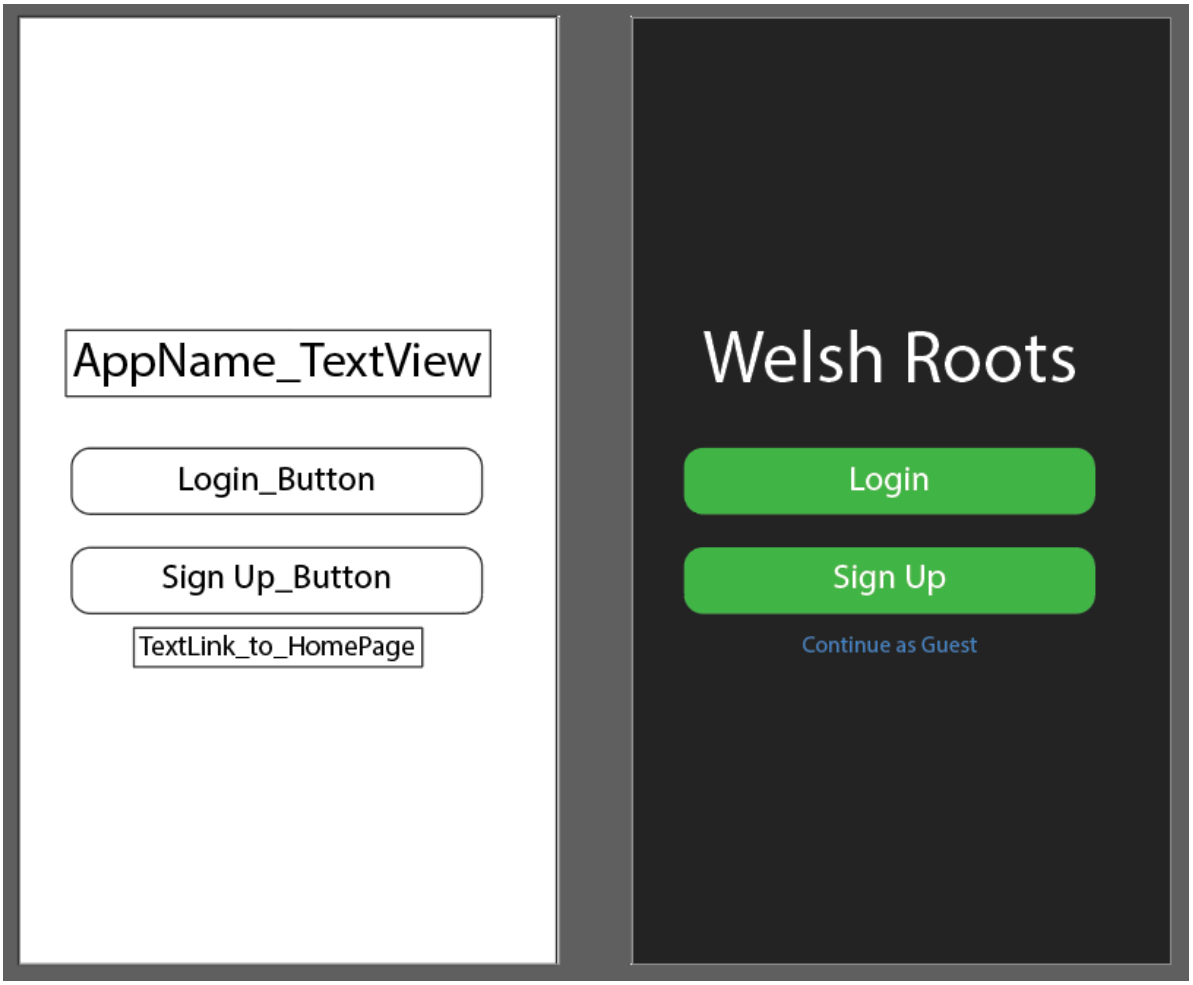
experience with the app, meeting the interactive feature requirement. To satisfy the need for a content library, the app includes articles discussing various aspects of Welsh heritage, such as urban legends and folklore and Welsh history. All written content in the app is available in both Welsh and English, and users can toggle the language settings within the app or adjust their device settings to switch languages. For the events and news section, the app includes an article highlighting special Welsh events and their respective dates, keeping users informed about upcoming cultural activities. Regarding the accessibility of the app for individuals with disabilities, the app includes screen reader support, enabling visually impaired users to navigate and interact with the app's features through audio feedback. Additionally, the app allows users to adjust the text size, catering to those with visual impairments or reading difficulties by providing customizable font sizes for better readability. These features ensure that the app remains inclusive and accessible to a broader audience, aligning with best practices for accessibility in mobile application design.

Proposed Design and Architecture

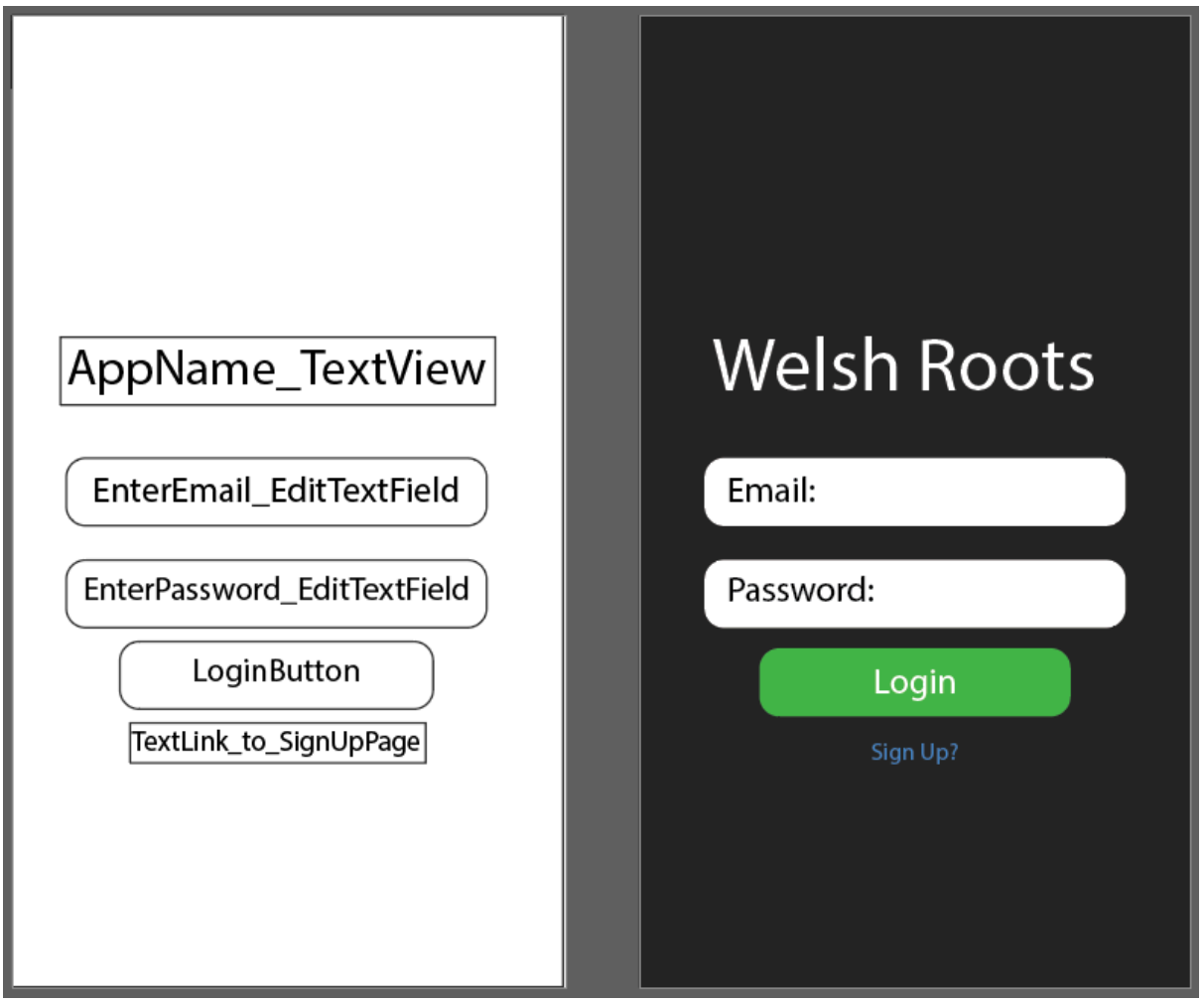
Wireframes:



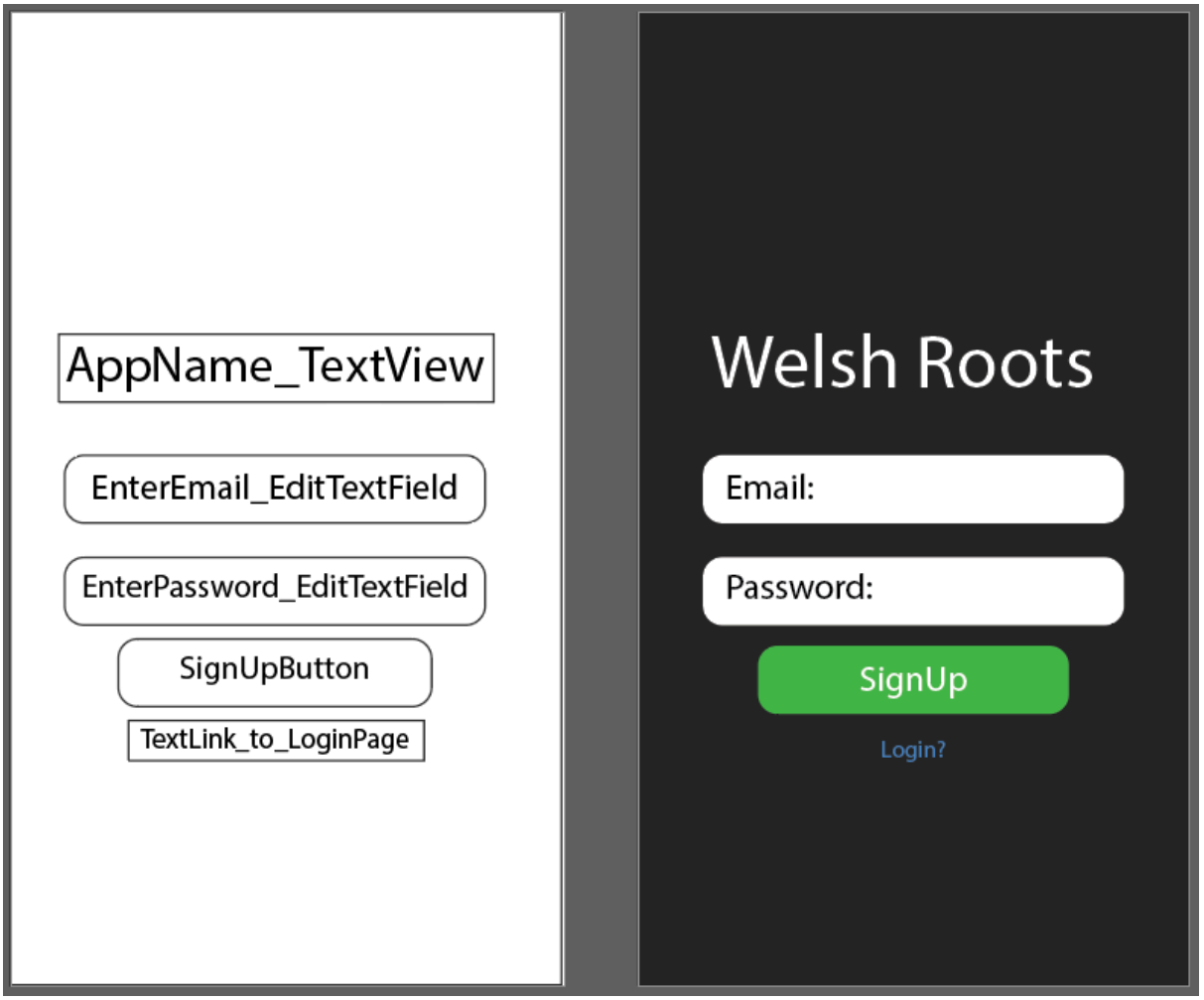
This is the design for the splash screen. The Splash screen contains the Welsh Roots app logo and is present for a couple of seconds after the Welsh Roots app is initialised.



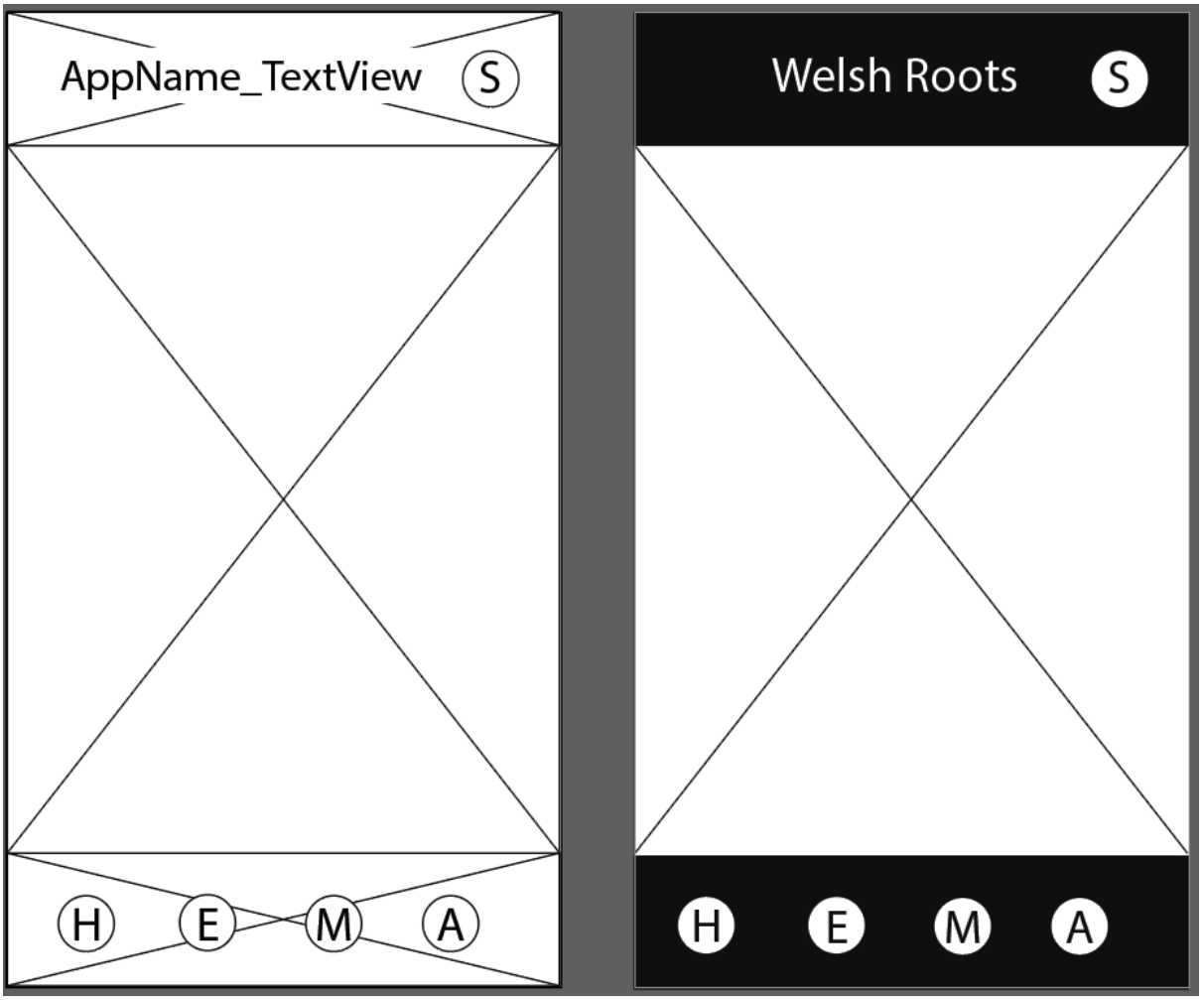
This is the design for the Welcome View. The user will be prompted to login, sign up or continue as a guest. If the user decides to select the login button, they will be taken to the login view. If the user selects the sign-up button, they will be taken to the sign-up view. If the user selects the continue as guest text link, they will be taken straight to the dashboard view (Home View).



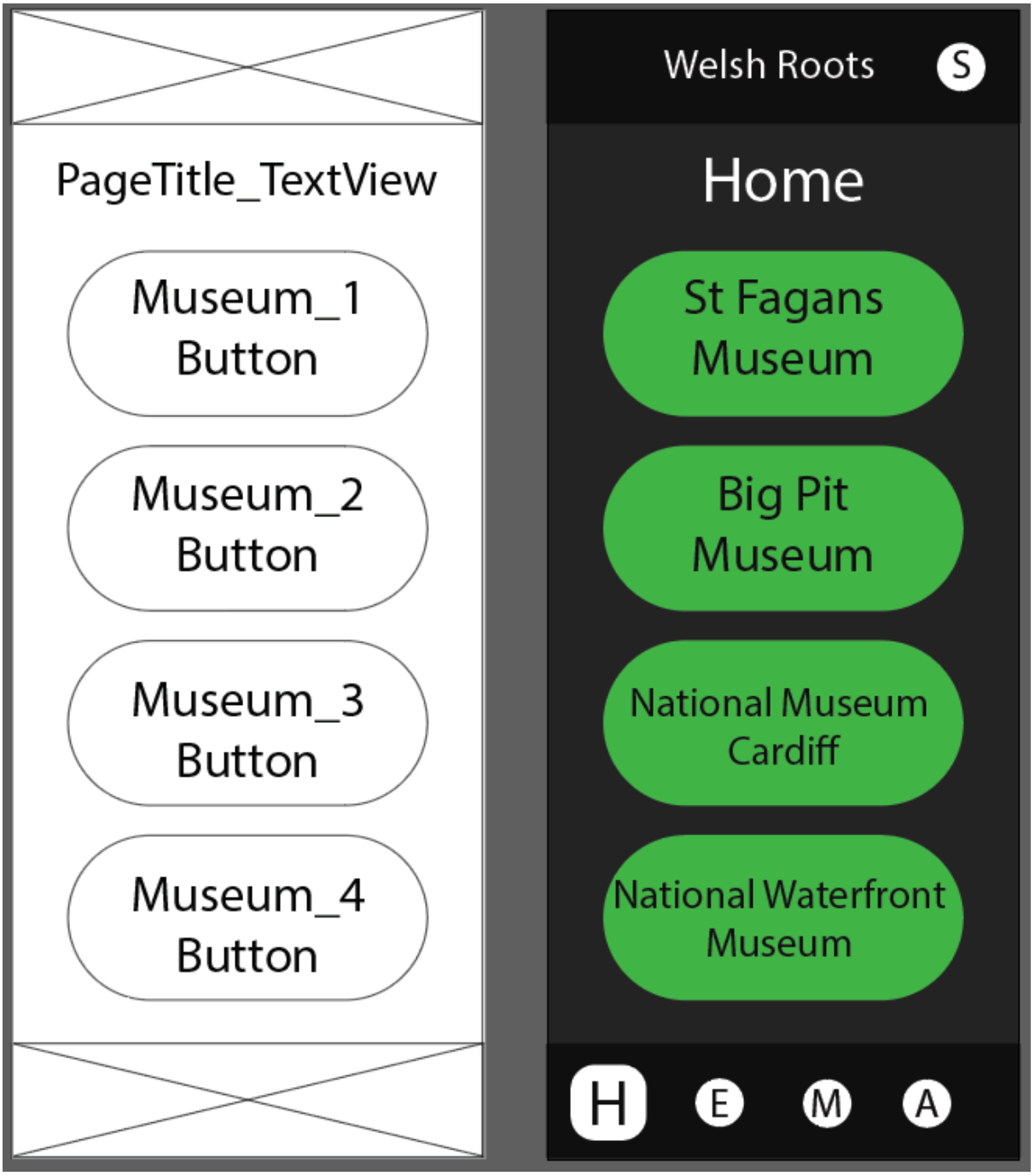
This design showcases the login view. The user will be prompted to enter the email and password associated with their account. The user will also be able to navigate to the sign-up view if they select the sign-up text link.



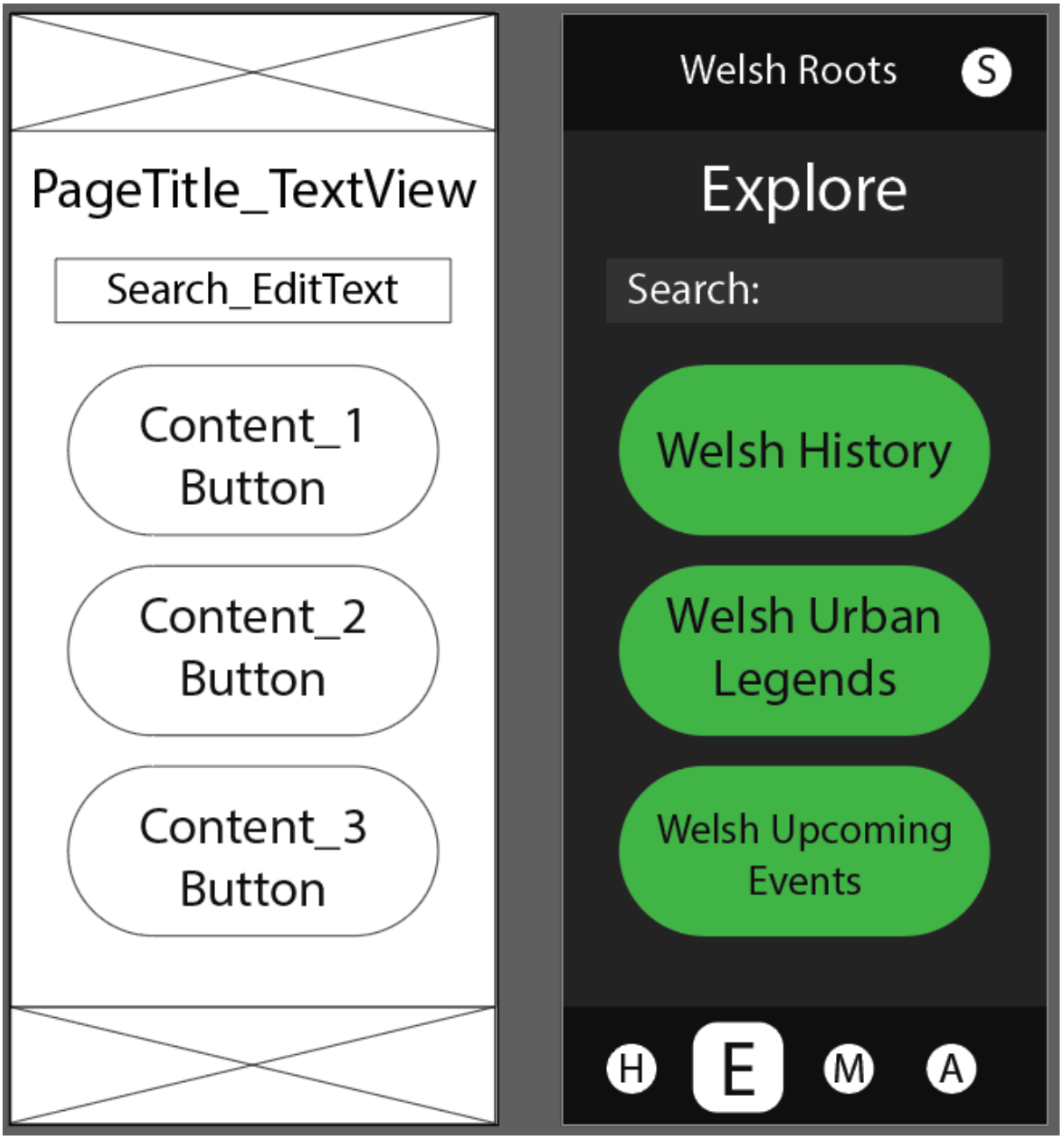
This design showcases the sign-up view. The user will be able to enter their email address and password that they want to use to register for an account. The user will be able to navigate to the login page by selecting the login text link.



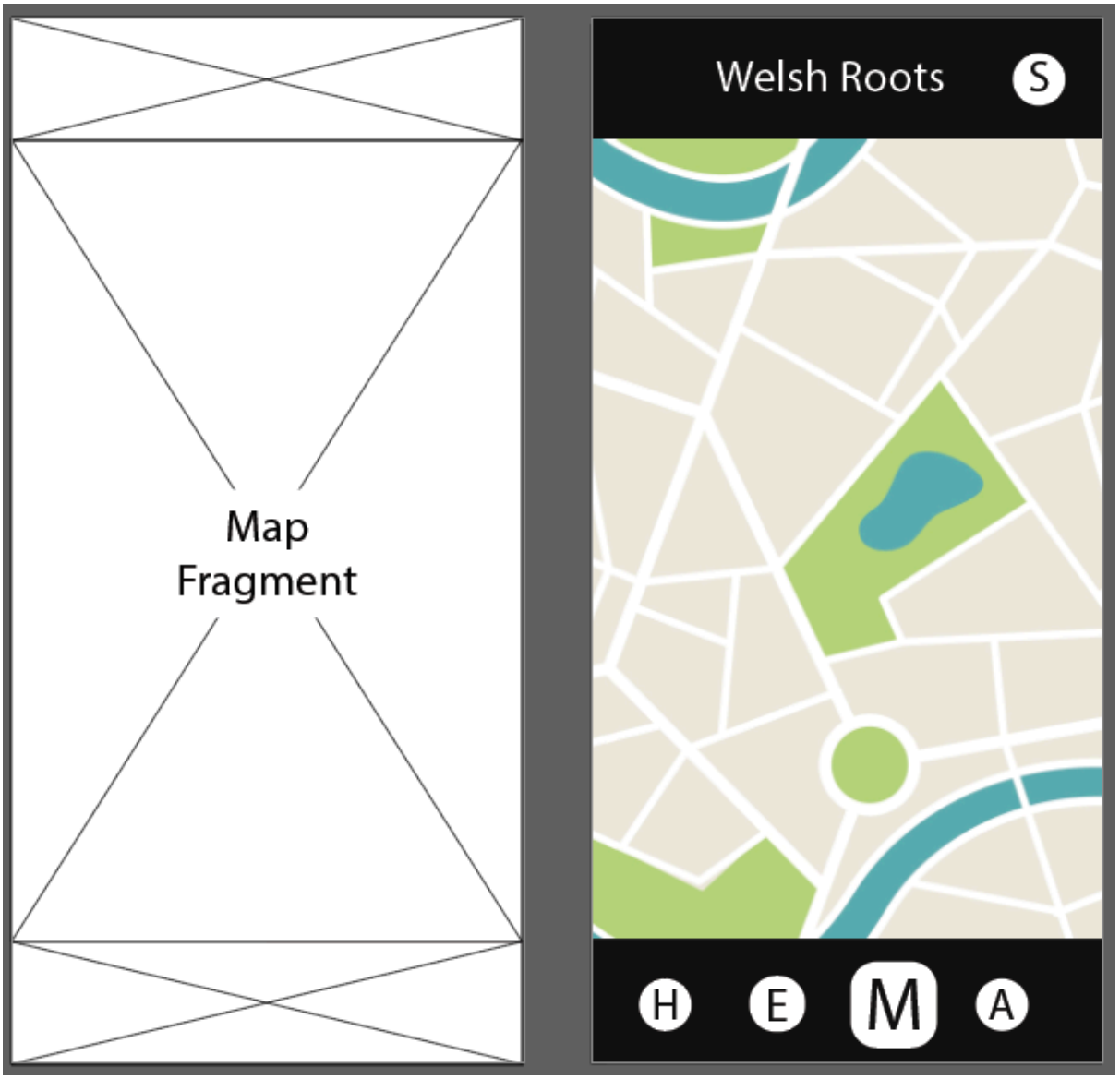
This wireframe showcases the dashboard fragment. This will display the banner and navigation bar on all views after the user successfully logs in or continues as a guest. The banner will display the name of the app and a settings button. Once the settings button is selected the user will be directed to the settings view. The navigation bar contains buttons to the home view, explore view, map view and about view. Once selected these buttons will take the user to their respective fragment views.



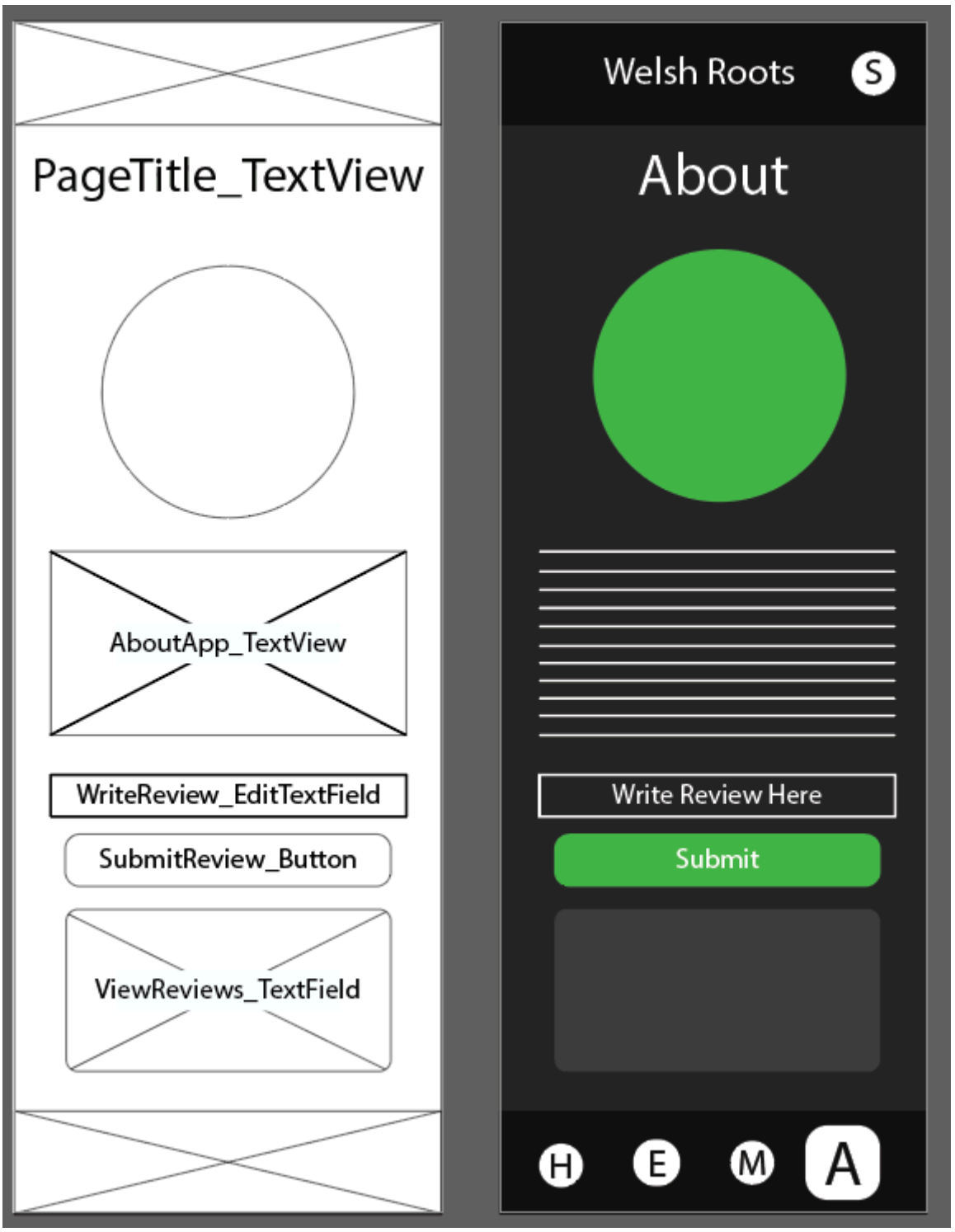
This design showcases the home view. On this, the user can choose to view the museum content of specific museums. The content includes information and exhibits from their respective museums. Big Pit Museum will showcase exhibitions and information specifically centred around the Big Pit Museum, etc. The user will be able to navigate through the app using the navigation bar at the bottom, the settings button at the top and the museum buttons.



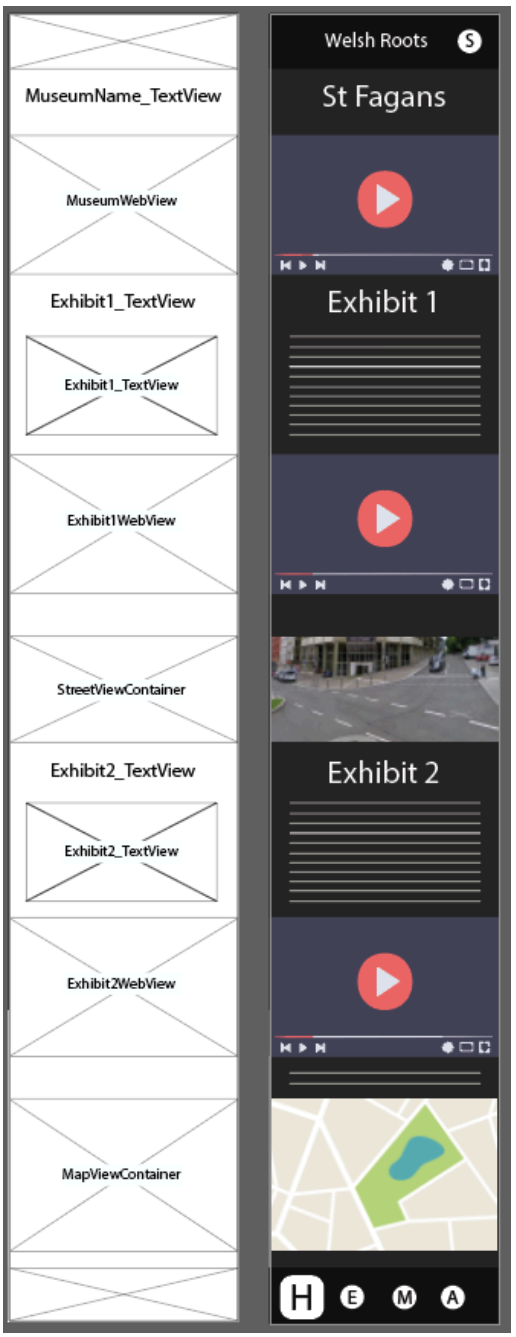
This wireframe explores the design of the explore view. The user will be able to view content centred around Wales. This view is the content library of the app, and the user can navigate through. The user will be able to utilise the search bar to find specific content.



This design shows the map view. Thie user will be able to view google maps within the Welsh Roots app. The map view will contain markers showcasing Welsh Museums that are included in the app that the user can select and see the name of the museum. The user will be able to manoeuvre the map around allowing for a wide coverage of possible locations.



This design showcases the about view. In this view the user will be able to read a short paragraph about the app and its purpose. The user will also be able to leave a review about the app and their experience with it.



This design showcases the view of the museum content. Each museum showcase: St. Fagans, Big Pit, National Museum Cardiff and National Waterfront Museum will follow the same structure. A virtual tour in the form of a YouTube video, the title of the first exhibit, a short paragraph detailing the exhibit, another YouTube video centred around exhibit 1. Then google street view located at the museum being showcased. After that the user will be able to see the next exhibit following the same structure as exhibit 1. Then after that the user will be able to see the location of the museum being showcased in the form of a marker placed in a google maps container.

Development Process

Implementation:

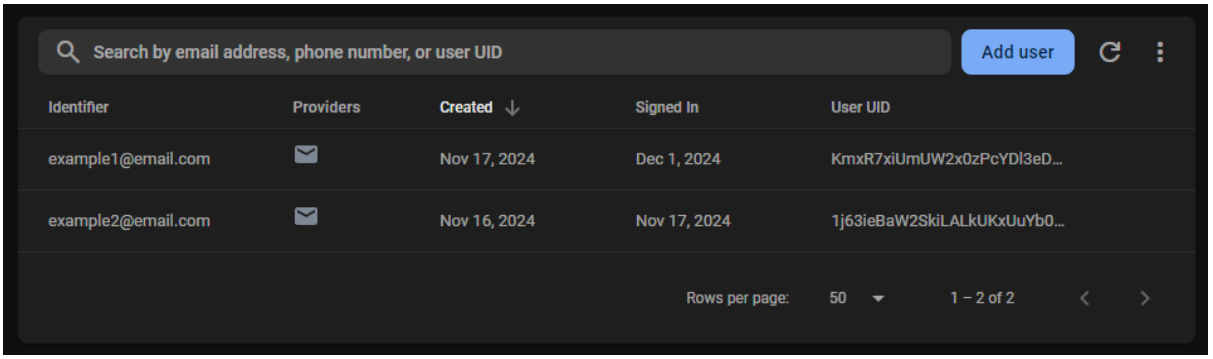
Sign-Up Implementation:

To implement the sign-up feature in my Welsh Roots app, I used Firebase Authentication. The SignUpActivity includes an intuitive user interface with fields for email and password input, a sign-up button, and a link to the login screen. Input validation is performed to ensure the password meets the required format and criteria (minimum password length of six characters).

```
// Sign Up Button
buttonSignUp.setOnClickListener(v -> {
    String email = editTextEmail.getText().toString().trim();
    String password = editTextPassword.getText().toString().trim();
    // Validate Email
    if (TextUtils.isEmpty(email)) {
        editTextEmail.setError("Email is required");
        return;
    }
    // Validate Password
    if (TextUtils.isEmpty(password)) {
        editTextPassword.setError("Password is required");
        return;
    }
    // Validate Password Length
    if (password.length() < 6) {
        editTextPassword.setError("Password must be at least 6 characters");
        return;
    }
```

Upon successful validation, the app uses Firebase's createUserWithEmailAndPassword() method to create a new user account. If the operation is successful, the user is redirected to the login screen with a success message displayed via a Toast. If an error occurs, the app displays the specific error message, ensuring clear feedback for the user.

```
// Create User with Firebase Authentication
firebaseAuth.createUserWithEmailAndPassword(email, password)
        .addOnCompleteListener( activi SignUpActivity.this, task -> {
            if (task.isSuccessful()) {
                Toast.makeText( conte SignUpActivity.this,  text "Sign-Up Successful!", Toast.LENGTH_SHORT).show();
                startActivity(new Intent( packageCor SignUpActivity.this, LoginActivity.class));
                finish();
            } else {
                Toast.makeText( conte SignUpActivity.this,  text "Error: " + Objects.requireNonNull(task.getException()).getMessage(), Toast.LENGTH_SHORT).show();
            }
        });
```



The activity also features bilingual support, with all labels and messages easily translatable into Welsh and English. This implementation ensures a secure and accessible user experience while integrating seamlessly with Firebase services.

Login Implementation:

The LoginActivity in the Welsh Roots app is responsible for handling user login using Firebase Authentication. It consists of two main sections email and password input fields, and buttons for login and sign-up. Upon launching, the activity initialises the necessary UI elements, including the email and password EditText fields, the login Button, and a TextView for navigating to the sign-up screen. When the user clicks the login button, the app first validates that the email and password fields are not empty, and ensures that the password meets the minimum length requirement (at least 6 characters). If any of these conditions are not met, the appropriate error messages are displayed.

```
// Login Button
buttonLogin.setOnClickListener(v -> {
    // Retrieve Input From EditText Field
    String email = editTextEmail.getText().toString().trim();
    String password = editTextPassword.getText().toString().trim();
    // Validate Email Field
    if (TextUtils.isEmpty(email)) {
        editTextEmail.setError("Email is required");
        return;
    }
    // Validate Password Field
    if (TextUtils.isEmpty(password)) {
        editTextPassword.setError("Password is required");
        return;
    }
    // Password Length Validation
    if (password.length() < 6) {
        editTextPassword.setError("Password must be at least 6 characters");
        return;
    }
```

If the inputs are valid, the app attempts to log the user in using Firebase Authentication's signInWithEmailAndPassword method. If the login is successful, the user is redirected to the DashboardActivity, and a success message is shown. If the login fails, an error message with the specific exception is displayed.

```
    // Sign In Using Firebase Authentication
    firebaseAuth.signInWithEmailAndPassword(email, password)
            .addOnCompleteListener( activity: LoginActivity.this, task -> {
                if (task.isSuccessful()) {
                    Toast.makeText( context: LoginActivity.this,  text: "Login Successful!", Toast.LENGTH_SHORT).show();
                    startActivity(new Intent( packageContext: LoginActivity.this, DashboardActivity.class));
                    finish();
                } else {
                    Toast.makeText( context: LoginActivity.this,  text: "Error: " + Objects.requireNonNull(task.getException()).getMessage(), Toast.LENGTH_SHORT).show();
                }
            });
});
```

Additionally, the TextView for sign-up provides a way for users who do not have an account to navigate to the SignUpActivity to create one. This flow ensures that users can securely log in or create an account.

Explore Implementation:

The ExploreFragment in the Welsh Roots app provides users with a convenient way to explore Welsh history, urban legends, and news. It includes three buttons—each linked to a respective fragment: WelshHistoryFragment, WelshUrbanLegends, and WelshNews. When clicked, these buttons navigate the user to the corresponding fragment, with the transaction added to the back stack for easy navigation back.

```
    buttonWelshHistory = rootView.findViewById(R.id.buttonWelshHistoryContent);
    buttonWelshUrbanLegends = rootView.findViewById(R.id.buttonWelshUrbanLegends);
    buttonWelshNews = rootView.findViewById(R.id.buttonWelshNews);

    // Welsh History Button
    buttonWelshHistory.setOnClickListener(v ->
        getParentFragmentManager() FragmentManager
                .beginTransaction() FragmentTransaction
                .replace(R.id.nav_host_fragment, new WelshHistoryFragment())
                .addToBackStack( name: null)
                .commit()
    );

    // Welsh Urban Legends Button
    buttonWelshUrbanLegends.setOnClickListener(v ->
        getParentFragmentManager() FragmentManager
                .beginTransaction() FragmentTransaction
                .replace(R.id.nav_host_fragment, new WelshUrbanLegends())
                .addToBackStack( name: null)
                .commit()
    );

    // Welsh News Button
    buttonWelshNews.setOnClickListener(v ->
        getParentFragmentManager() FragmentManager
                .beginTransaction() FragmentTransaction
                .replace(R.id.nav_host_fragment, new WelshNews())
                .addToBackStack( name: null)
                .commit()
    );
```

Additionally, the fragment features a SearchView that allows users to filter the buttons dynamically based on the text entered. As users type, the search function hides or displays the relevant buttons based on whether the query matches predefined labels like "welsh history," "welsh urban legends," or "welsh news." This setup provides a user-friendly and interactive way to explore Welsh-themed content while making navigation efficient with the added search functionality.

```
    // Search Function
    SearchView searchView = rootView.findViewById(R.id.searchView);
    searchView.setOnQueryTextListener(new SearchView.OnQueryTextListener() {
        1 usage
        @Override
        public boolean onQueryTextSubmit(String query) { return false; }

        1 usage
        @Override
        public boolean onQueryTextChange(String newText) {
            filterContent(newText);
            return false;
        }
    });
    return rootView;
}
```

Dashboard Implementation:

In the Welsh Roots app, the DashboardActivity serves as the central hub for navigation. The activity utilises a BottomNavigationView to provide a way to switch between different fragments, such as: Home, Explore, Maps, About, and Settings. By default, the app loads the HomeFragment, creating a familiar starting point for users. Each navigation item dynamically loads its corresponding fragment using the loadFragment() method, ensuring smooth transitions without reloading the entire activity. Additionally, a settings icon in the top-right corner allows users to quickly access the SettingsFragment.

```
// Set View
setContentView(R.layout.activity_dashboard);
// Bottom Navigation Widget
BottomNavigationView bottomNavigationView = findViewById(R.id.bottom_navigation);
// Load Home By Default
loadFragment(new HomeFragment());

bottomNavigationView.setOnItemSelectedListener(item -> {
    Fragment selectedFragment = null;

    // Load Selected Navigation Item
    if (item.getItemId() == R.id.nav_home) {
        selectedFragment = new HomeFragment();
    } else if (item.getItemId() == R.id.nav_explore) {
        selectedFragment = new ExploreFragment();
    } else if (item.getItemId() == R.id.nav_maps) {
        selectedFragment = new MapsFragment();
    } else if (item.getItemId() == R.id.nav_about) {
        selectedFragment = new AboutFragment();
    }

    if (selectedFragment != null) {
        loadFragment(selectedFragment);
    }
    return true;
});
```

The activity applies the user's saved theme preferences, stored in SharedPreferences, to maintain a consistent light or dark mode experience. This feature ensures the app adheres to user preferences every time it is launched.

```
// Retrieve Current Theme Preferences
SharedPreferences prefs = getSharedPreferences( name: "ThemePrefs", MODE_PRIVATE);
int currentMode = prefs.getInt( key: "theme", AppCompatDelegate.MODE_NIGHT_NO);
AppCompatDelegate.setDefaultNightMode(currentMode); // Apply the theme
```

Maps Feature:

The MapsFragment in the Welsh Roots app integrates Google Maps functionality to allow users to explore the locations of museums in Wales. This feature provides an interactive map view with markers representing the museums showcased in the app, enhancing the app's ability to engage users with Welsh heritage. The onCreateView method inflates the layout for the fragment and initialises the map component using a SupportMapFragment. The getChildFragmentManager() is utilized to locate the map fragment within the current view, and the getMapAsync method is called to ensure that the map is ready before performing any operations on it.

```
@Override
public View onCreateView(LayoutInflater inflater, ViewGroup container, Bundle savedInstanceState) {
    View view = inflater.inflate(R.layout.fragment_maps, container,  attachToRoot: false);

    SupportMapFragment mapFragment =
            (SupportMapFragment) getChildFragmentManager().findFragmentById(R.id.mapFragment);
    if (mapFragment != null) {
        mapFragment.getMapAsync( callback: this);
    }
    return view;
}
```

When the map is ready, the onMapReady method is invoked, where a list of museum locations is defined using a custom MarkerData class. Each location consists of latitude and longitude coordinates (LatLng) and a title. These markers are added to the map using the addMarker method of the GoogleMap class. The camera's initial position is set to the first museum's location using moveCamera and CameraUpdateFactory.newLatLngZoom, providing a focused view for users upon launching the map.

```
    // List of Marker Data
    List<MarkerData> markerDataList = new ArrayList<>();
    // Marker Data
    markerDataList.add(new MarkerData(new LatLng( latitude: 51.48722473313164,  longitude: -3.2723620534115585),  title: "St Fagans National Museum of History"));
    markerDataList.add(new MarkerData(new LatLng( latitude: 51.77256387585435,  longitude: -3.1045690206749548),  title: "Big Pit National Coal Museum"));
    markerDataList.add(new MarkerData(new LatLng( latitude: 51.485772068873864,  longitude: -3.176852611411994),  title: "National Museum Cardiff"));
    markerDataList.add(new MarkerData(new LatLng( latitude: 51.616701290235795,  longitude: -3.9391617532728214),  title: "National Waterfront Museum"));

    // Add Marker to Map
    for (MarkerData markerData : markerDataList) {
        googleMap.addMarker(new MarkerOptions()
                .position(markerData.latLng)
                .title(markerData.title));
    }

    // Set Zoom Length and Move Camera View to First Marker
    if (!markerDataList.isEmpty()) {
        googleMap.moveCamera(com.google.android.gms.maps.CameraUpdateFactory.newLatLngZoom(markerDataList.get(0).latLng,  zoom: 11));
    }
}
```

The modular design of the MapsFragment ensures that additional locations can be easily added in the future.

Showcasing Museum Features:

The MuseumStFagans fragment in the Welsh Roots app provides an interactive and immersive experience for users interested in learning more about St Fagans National Museum of History. This fragment integrates multiple media and map features to deliver an immersive exploration of the museum. In the onCreateView method, the fragment's layout is inflated, and several interactive components are initialized. First, YouTube videos are embedded using WebView elements to showcase a virtual tour of the museum and videos centred around 2 exhibits. Each WebView is configured to enable JavaScript, and the respective YouTube video URLs are loaded using the video IDs for St Fagans and the exhibits. This setup allows users to view the videos directly within the app without leaving the interface.

```
// St Fagans Youtube Video
WebView stFagansYoutube = view.findViewById(R.id.stFagansYoutube);
WebSettings webSettingsStFagansVideo = stFagansYoutube.getSettings();
webSettingsStFagansVideo.setJavaScriptEnabled(true);
stFagansYoutube.loadUrl("https://www.youtube-nocookie.com/embed/" + StFagansVideoID);
stFagansYoutube.setWebViewClient(new WebViewClient());
```

Additionally, a SupportStreetViewPanoramaFragment is used to integrate a street view of the museum. This fragment is added to the layout dynamically, and its position is set to the St Fagans location on the map, offering users a 360-degree virtual tour of the museum environment.

```
// St Fagans Exhibit Street View
SupportStreetViewPanoramaFragment streetViewFragment = new SupportStreetViewPanoramaFragment();
FragmentTransaction streetViewTransaction = getChildFragmentManager().beginTransaction();
streetViewTransaction.replace(R.id.streetViewContainer, streetViewFragment);
streetViewTransaction.commit();
streetViewFragment.getStreetViewPanoramaAsync(panorama ->
        panorama.setPosition(StFagansLocation)
);
```

A SupportMapFragment is also included, displaying a map with a marker for the museum's location. The map is configured to zoom in on the St Fagans location, providing users with an easy way to locate the museum and explore its surroundings.

```
    // St Fagans Map Location
    SupportMapFragment mapFragment = new SupportMapFragment();
    FragmentTransaction mapTransaction = getChildFragmentManager().beginTransaction();
    mapTransaction.replace(R.id.mapContainer, mapFragment);
    mapTransaction.commit();
    mapFragment.getMapAsync(googleMap -> {
        LatLng StFagansLocation = new LatLng( latitude: 51.4871873597198,  longitude: -3.272341703519731);
        googleMap.addMarker(new MarkerOptions().position(StFagansLocation).title("St.Fagans National Museum of History"));
        googleMap.moveCamera(com.google.android.gms.maps.CameraUpdateFactory.newLatLngZoom(StFagansLocation,  zoom: 15));
    });
    return view;
}
```

Review Feature:

The review functionality in the AboutFragment of the Welsh Roots app allows users to submit and view reviews within the app. It consists of an EditText for inputting reviews, a Button to submit the review, and a TextView to display the accumulated reviews. When the fragment is created, the loadReviews method retrieves any previously saved reviews from SharedPreferences and displays them. If no reviews exist, it shows "No Reviews at the Moment." When a user submits a review, the app checks if the input field is empty; if not, it saves the review to SharedPreferences and updates the display with the new review. If the field is empty, a toast message notifies the user that no review was submitted. The saveReview method appends new reviews to existing ones, ensuring that all reviews are preserved and displayed together in the app, offering a simple yet effective way for users to interact with and share their feedback.

```
    // Load Reviews
    loadReviews();

    // Submit Button
    submitReviewButton.setOnClickListener(v -> {
        String reviewText = reviewInput.getText().toString().trim();

        // Check if Review is Empty
        if (TextUtils.isEmpty(reviewText)) {
            Toast.makeText(getContext(), text: "No Review Submitted", Toast.LENGTH_SHORT).show();
        } else {
            saveReview(reviewText);
            reviewInput.setText(""); // Clear input field
            Toast.makeText(getContext(), text: "Review Submitted Successfully", Toast.LENGTH_SHORT).show();
        }
    });
    return view;
}
```

Settings Features:

The SettingsFragment in the Welsh Roots app provides users with the option to adjust the app's theme and language settings. It includes two switches, one for toggling between the light and dark mode.

```
// Retrieve Current Theme
SharedPreferences prefs = requireActivity().getSharedPreferences( name: "ThemePrefs", Context.MODE_PRIVATE);
int currentMode = prefs.getInt( key: "theme", AppCompatDelegate.MODE_NIGHT_NO);
// Set Switch Position to Saved Theme
themeSwitch.setChecked(currentMode == AppCompatDelegate.MODE_NIGHT_YES);
// Change Theme when Switch is Changed
themeSwitch.setOnCheckedChangeListener((buttonView, isChecked) -> {
    SharedPreferences.Editor editor = prefs.edit();
    if (isChecked) {
        // Dark Mode
        AppCompatDelegate.setDefaultNightMode(AppCompatDelegate.MODE_NIGHT_YES);
        editor.putInt("theme", AppCompatDelegate.MODE_NIGHT_YES);
    } else {
        // Light Mode
        AppCompatDelegate.setDefaultNightMode(AppCompatDelegate.MODE_NIGHT_NO);
        editor.putInt("theme", AppCompatDelegate.MODE_NIGHT_NO);
    }
    // Save Selected Theme
    editor.apply();
});
```

Then another for switching between the English and Welsh languages.

```
    // Switch Language
    @SuppressLint("UseSwitchCompatOrMaterialCode") Switch languageSwitch = view.findViewById(R.id.modeLanguage);
    // Retrieve Current Language
    SharedPreferences langPrefs = requireActivity().getSharedPreferences( name: "LanguagePrefs", Context.MODE_PRIVATE);
    String currentLanguage = langPrefs.getString( key: "language",  defValue: "en");
    // Set Switch Position to Saved Language
    languageSwitch.setChecked(currentLanguage.equals("cy"));
    // Change Language when Switch is Changed
    languageSwitch.setOnCheckedChangeListener((buttonView, isChecked) -> {
        String newLanguage = isChecked ? "cy" : "en";
        setLocale(newLanguage);
        // Save Selected Language
        SharedPreferences.Editor langEditor = langPrefs.edit();
        langEditor.putString("language", newLanguage);
        langEditor.apply();
        requireActivity().recreate();
    });
    return view;
}
```

Upon loading, the fragment checks SharedPreferences to retrieve the current settings for both theme and language, ensuring that the switches are correctly positioned according to the saved preferences.

For the theme, the app uses AppCompatDelegate to set either dark or light mode based on the saved preference. When the user toggles the theme switch, the app updates the theme and saves the new preference in SharedPreferences. When the user changes the language setting, the app switches between Welsh (cy) and English (en) by updating the app's locale and restarting the activity to apply the language change.

```
    // Method to Change Language
    1 usage
    private void setLocale(String langCode) {
        Locale locale = new Locale(langCode);
        Locale.setDefault(locale);
        Context context = requireActivity();
        android.content.res.Resources resources = context.getResources();
        android.content.res.Configuration config = resources.getConfiguration();
        config.setLocale(locale);
        resources.updateConfiguration(config, resources.getDisplayMetrics());
    }
}
```

This functionality ensures that the app offers a personalized experience based on user preferences, enhancing usability and accessibility for Welsh-speaking users.

The use of SharedPreferences means that these settings persist between app sessions, allowing users to maintain their chosen configurations.

Challenges:

There were multiple challenges I faced while working on this project. One of the biggest issues was that my laptop could not run the emulator needed to test the code I had written. To solve this, I tried multiple approaches. First, I attempted a factory reset on my laptop, but it had little impact on the problem, apart from crossing one item off the process of elimination checklist. Then, I tried uninstalling and reinstalling the software multiple times, using different sources and installation settings, but that didn't work either, crossing another item off the list of possible solutions. After many attempts, I realized the issue was related to the HAXM installer, which stands for Hardware Accelerated Execution Manager from Intel. Every time I tried to install it, the installation would fail. After some research, I found out the problem was because Intel Virtualization Technology (VT-X) wasn't enabled on my laptop. HAXM requires VT-X to be enabled, so I had to access my laptop's BIOS settings (Basic Input/Output System Settings) to turn it on. However, even after enabling VT-X and trying again, the installation still failed. My laptop has an Intel processor, but it's important to note that if your device doesn't have an Intel processor, you won't be able to use Intel's HAXM technology. Since my laptop runs Windows, I attempted to use the Windows Hypervisor Platform instead, but that also failed. Next, I tried disabling two optional features in Windows, Virtual Machine Platform and Windows Hypervisor Platform. Once I turned these off and ran the emulator again, the HAXM installer was successfully installed, and I was finally able to get the emulator working. Unfortunately, my laptop isn't powerful enough to run the emulator efficiently. It was very slow, often took a long time to start, and would sometimes crash unexpectedly. At this point, I decided to use the university's computer facilities, but that didn't solve the problem either. The university computers seemed to have the same issue, as HAXM couldn't be installed on them, all computers I tried failed to run the emulator. Eventually, I found a solution by borrowing a family member's Android tablet, an Acer Iconia One 10 (model B3-A30). This tablet runs Android version 6.0 (Marshmallow) and was released in 2016, making it about eight years old now. While the tablet was slow at times and struggled with modern apps, it worked better for testing my app than the emulator. Despite the limitations of using an outdated device, I was able to test my written code effectively.

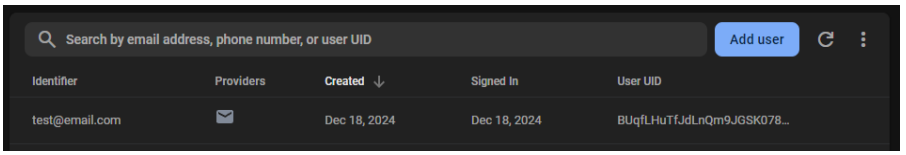
Another challenge I encountered during the development of the app was my idea of using custom map markers to showcase museums, historical sites, monuments, and other locations. The main difficulty was finding suitable icons to represent these locations and ensuring they stood out on the maps page. Although I initially implemented this feature, I later decided to remove it, as it ended up being more of a negative than a positive.

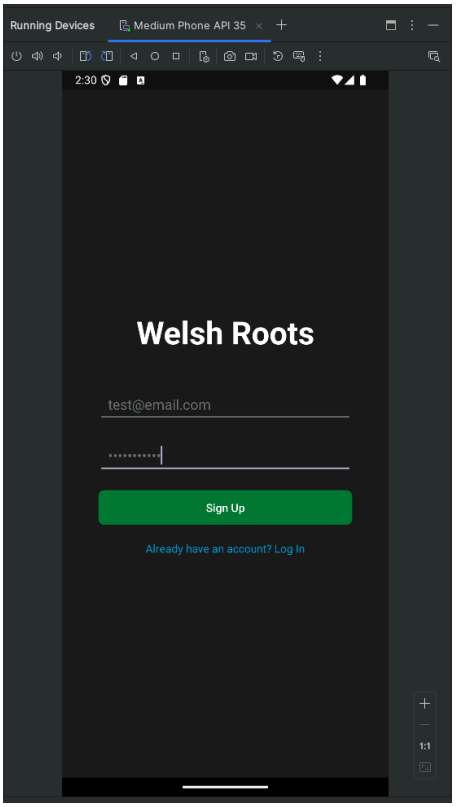
Testing:

Sign Up Feature:

Expectation – I expect the user to be able to get view the sign-up page, enter an email address, enter a password and be able to register for an account.

Result – Success

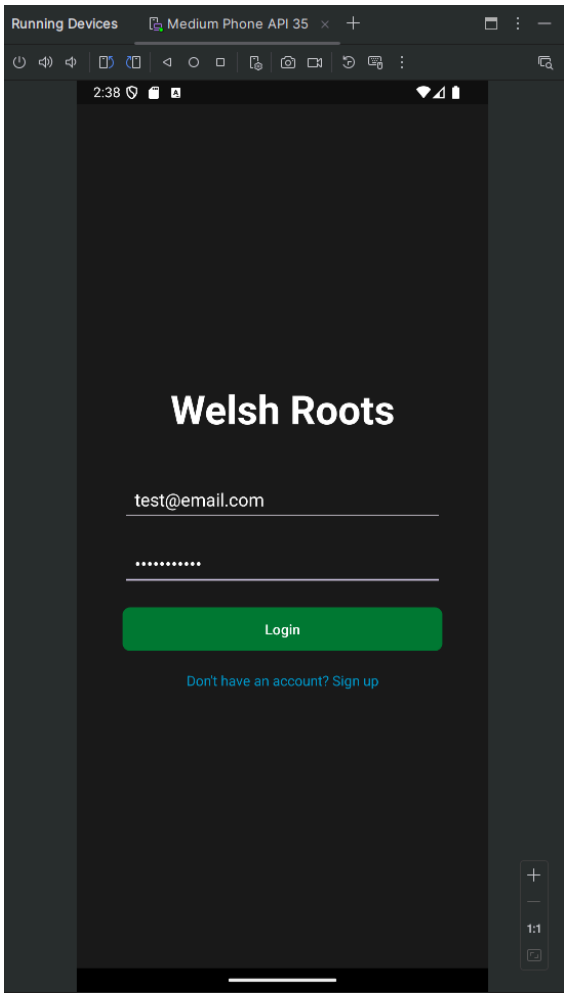


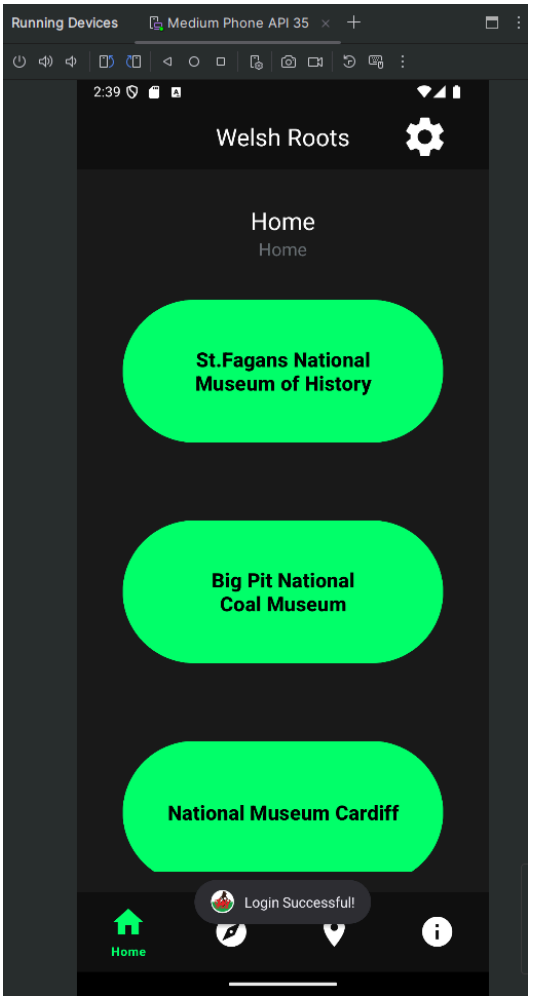


Login Feature:

Expectation – I expect the user to be able to enter in the email and password they used when signing up to login successfully.

Result - Success

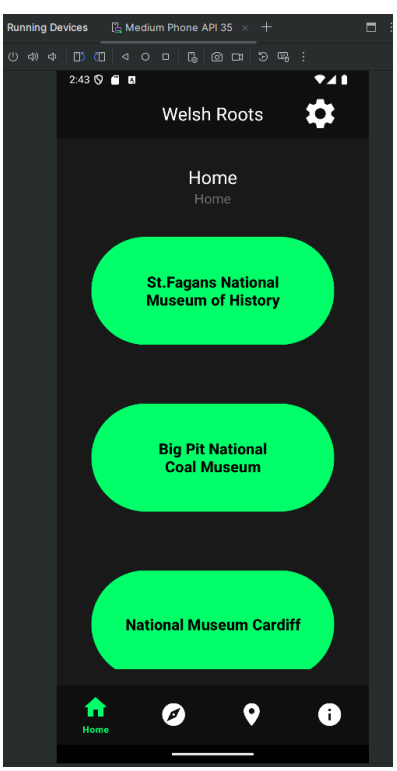


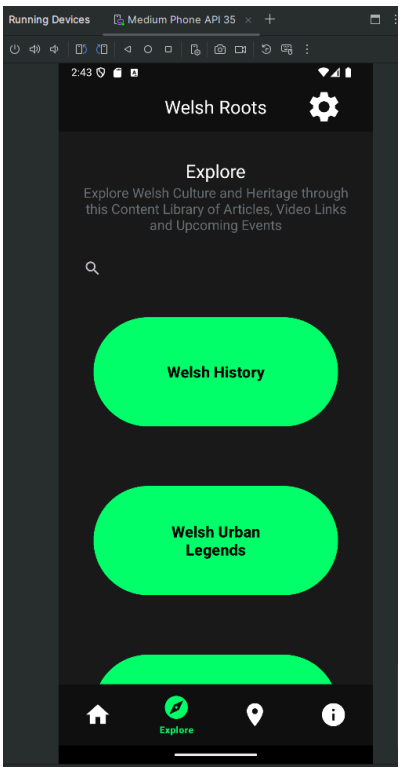


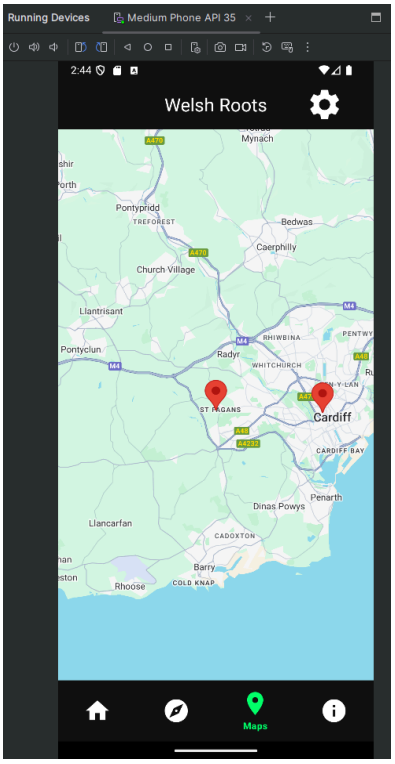
Dashboard Activity Feature:

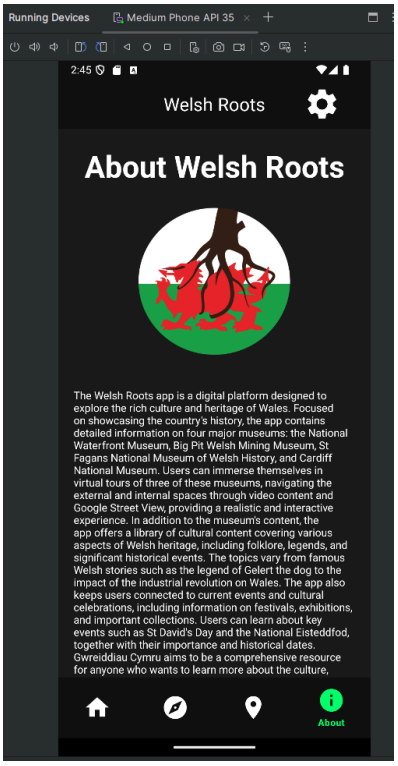
Expectation – I expect the app to display the banner and navigation bar on all views after the user either logs in successfully or continues as a guest.

Result – Success





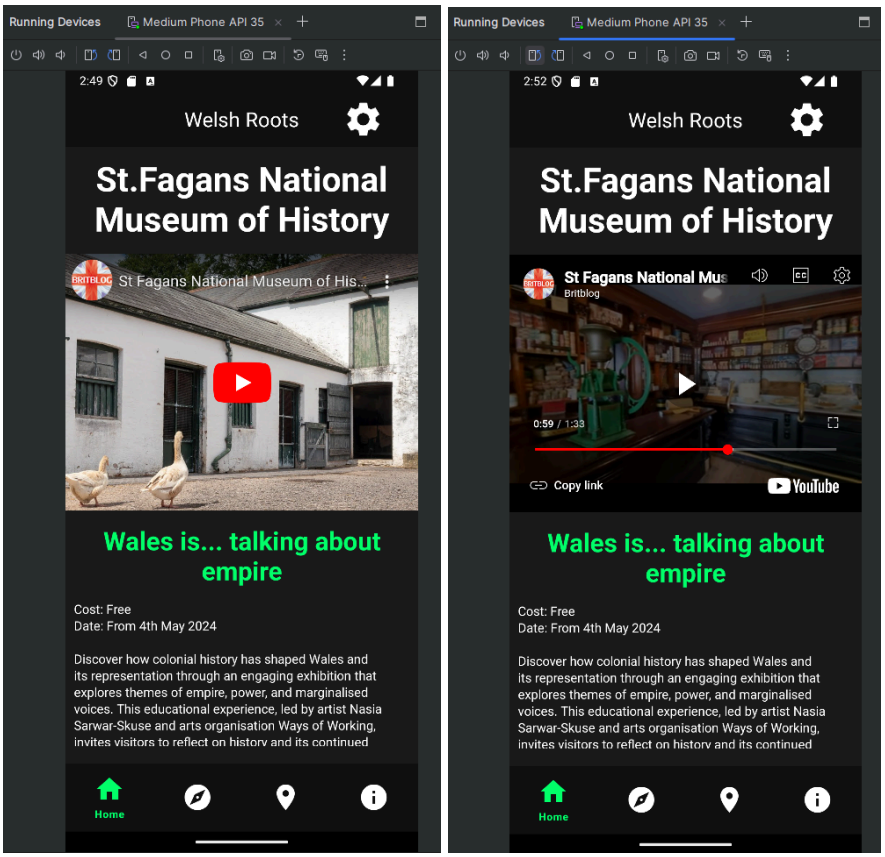




Virtual Tour Feature:

Expectation – I expect the app to showcase a virtual tour in the form of a YouTube video on the St. Fagans National Museum of History view. I also expect it to play the video in the app itself and not direct the user to the YouTube app in order to play the video.

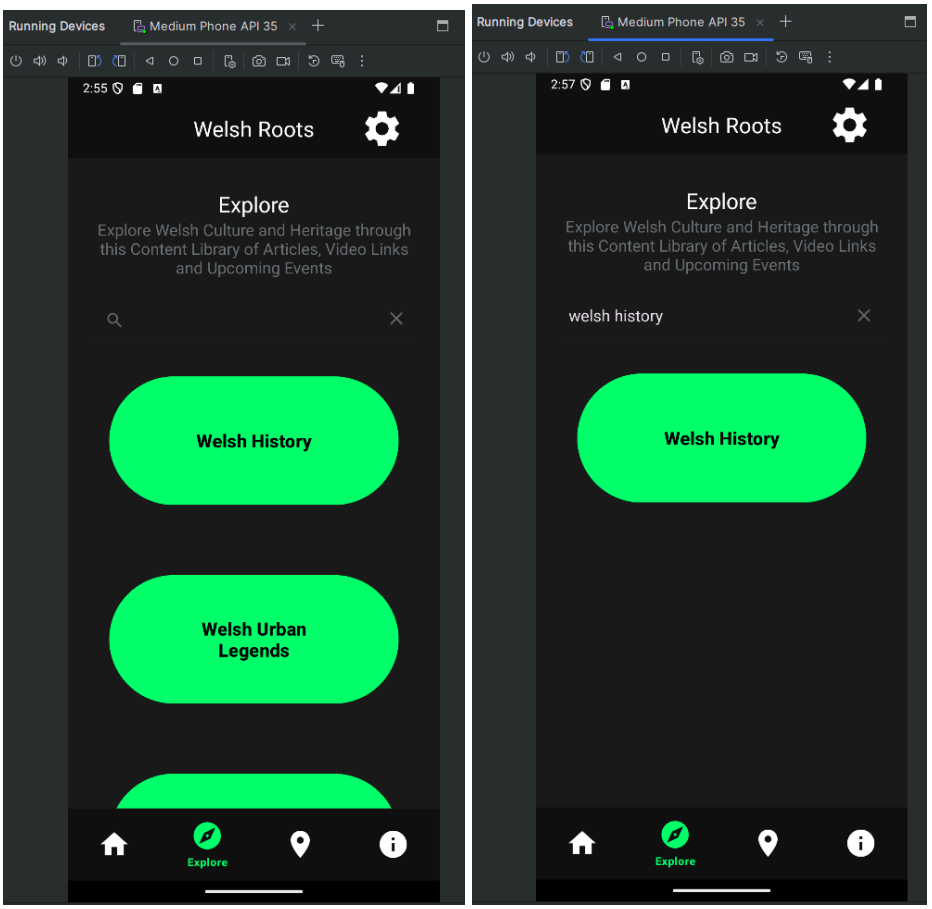
Result - Success



Search Feature:

Expectation – I expect all buttons to be shown until a user starts typing in the search bar, when the user finishes typing the search criteria, if that criterion matches a button in the content library, then that specific button should be shown to the user.

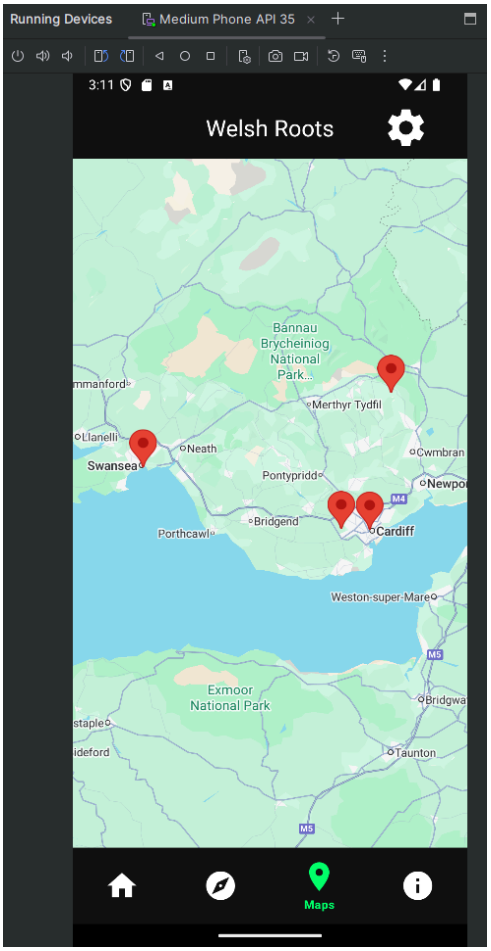
Result - Success



Map Feature:

Expectation – I expect the app to display the google maps view after the user navigates to the map view. I also expect 4 markers to be visible for: St Fagans, Big Pit, National Coal Museum and National Waterfront Museum.

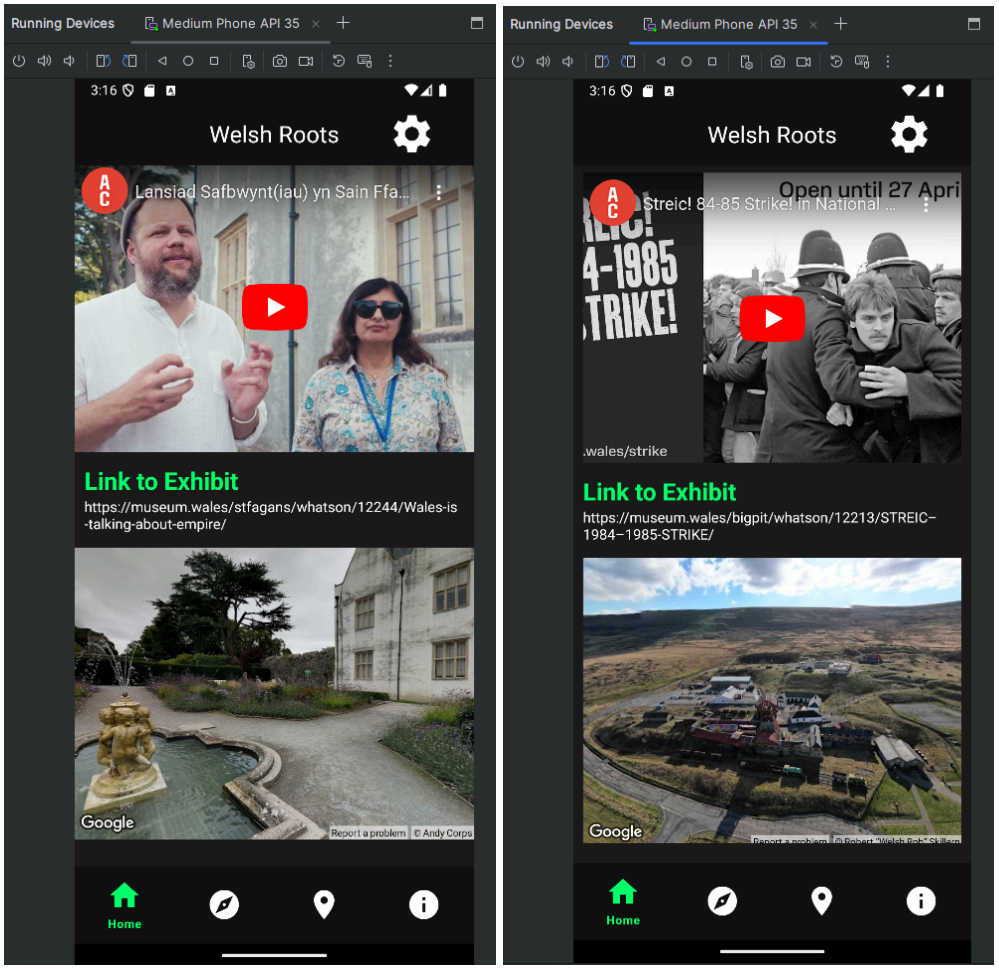
Result – Success



Google Street View Feature:

Expectation – I expect that on the museum content views, the app will display a container with google street view displayed and location of the street view to be the specific location of the museum being showcased.

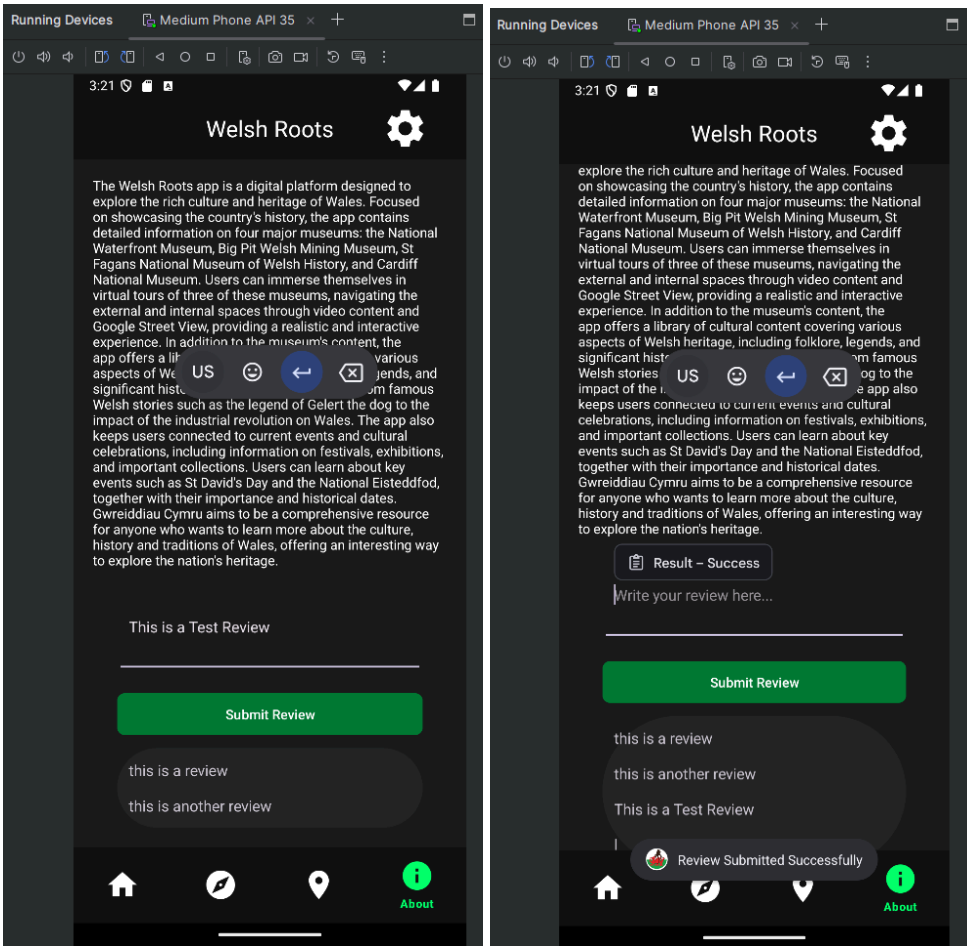
Result – Success



Review Feature:

Expectations – I expect the user to be able to type a review and submit it. The review should then be displayed for the user to view.

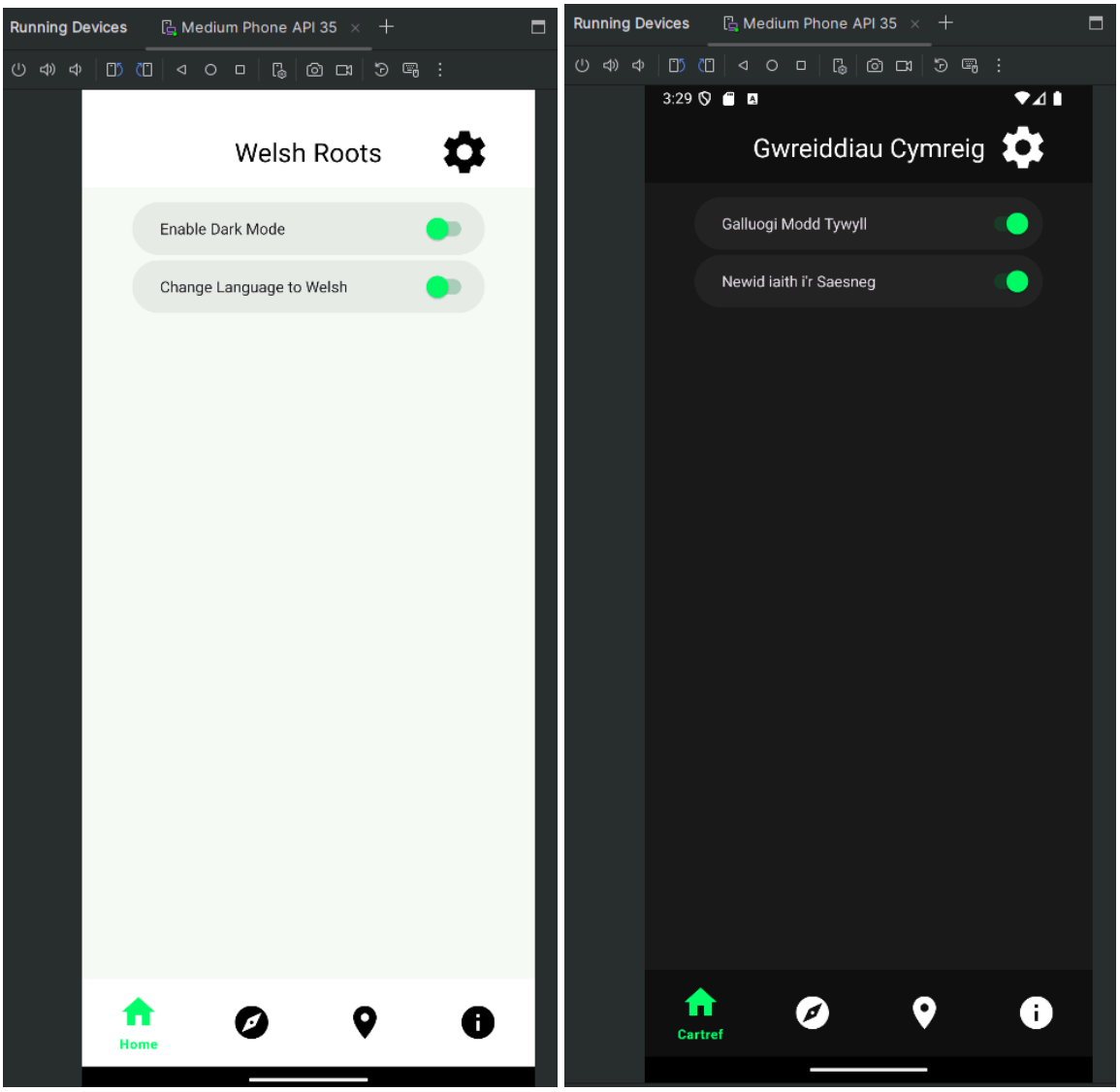
Result – Success



Settings Feature:

Expectations – I expect the user to be able to view the settings after navigating to the settings view. I also expect the user to be able to toggle between themes and languages. For themes I expect the user to be able to toggle between light mode and dark mode. For languages I expect the user to be able to toggle between English and Welsh. I also expect the user's preference to persist between app sessions.

Result - Success



Conclusion & Future Work

For potential improvements, the Welsh Roots app could benefit from expanding its content library to include more detailed information on Welsh culture, heritage, and history. This would make the app a more in depth resource for users interested in learning about various aspects of Wales. Additionally, adding interactive features such as quizzes, games, and challenges related to Welsh culture could enhance user engagement and make learning more enjoyable. Expanding the app’s language support to cover a broader range of languages, beyond just English and Welsh, would also make it more accessible to a global audience, attracting users from diverse backgrounds. Furthermore, the Welsh Roots app would benefit from partnering with Welsh museums, heritage sites, and/or cultural organizations. These collaborations could provide exclusive content and new features, strengthening the app's connection with Welsh culture. Finally, optimizing the app for a variety of devices, including tablets, phones, large screens and small screens, would increase its accessibility and usability, ensuring that it reaches as many users as possible. These improvements would help to making the Welsh Roots app a more engaging, inclusive, and educational platform for users worldwide.

References

Ceci, L (2024) *Language learning apps – statistics & facts.* Available at: https://www.statista.com/topics/8425/language-learning-apps/#topicOverview (Accessed: 12/2024)

Google (n.d.), *Google Maps Platform documentation*. Available at: https://developers.google.com/maps/documentation (Accessed: 11/2024).

Firebase. (2024) *Firebase Authentication documentation*. Available at: https://firebase.google.com/docs/auth (Accessed: 11/2024).